



مقدمه

در این تمرین قرار است با استفاده از الگوریتم‌های جستجوی آگاهانه و ناآگاهانه که در درس آموخته‌اید، راه حل مناسب برای مسئله‌ای که در ادامه مطرح می‌شود بیابید و آن را پیاده‌سازی کنید.

توضیح مسئله

در یک هزارتوی تاریک، ماری به دنبال سیب‌هایی که مخفی شده‌اند می‌گردد. مار که بسیار گرسنه‌ست و صبرش تمام شده، می‌خواهد تمام سیب‌ها را در سریع‌ترین زمان ممکن بخورد. اما باید حواسش را جمع کند تا به دیوارها و بدن خودش برخورد نکند که این کار با خوردن سیب‌های بیشتر و بلندتر شدن مار، سخت‌تر می‌شود. در این هزارتو حداکثر دو جفت پرتال قرار دارند که مار برای رسیدن سریع‌تر به سیب‌ها می‌تواند با ورود به یکی از آنان، از پرتال متناظرش خارج می‌شود.

حل این مسئله برایش دشوار است و از شما خواسته تا با الگوریتم‌های جستجو مسیری نشان او دهید.

ورودی و خروجی

ورودی برنامه یک نقشه است که مکان اجزای بازی روی آن مشخص است.

- S موقعیت شروع مار است.
- P و Q پرتال‌ها می‌باشند. اگر تنها یک جفت پرتال در نقشه حاضر باشد، Q نخواهد بود.
- A سیب را نشان می‌دهد.
- W دیوار را نشان می‌دهد.
- نقطه فضای خالی را برای حرکت مار نشان می‌دهد.
- با گذشت بازی و خوردن هر سیب توسط مار، یک B به انتهای بدنش اضافه خواهد شد.

• حرکت Toroidal: مار می‌تواند از لبه‌های نقشه عبور کند و از طرف مقابل ظاهر شود. به عنوان مثال، اگر مار از لبه راست خارج شود، از لبه چپ وارد می‌شود و بالعکس. همین رفتار برای لبه‌های بالا و پایین نیز صادق است.

خروجی برنامه یک دنباله از حرکات است که با شروع از نقشه‌ی بازی، مار تمام سیب‌ها را بدون نقض قوانین بازی بخورد.

نمونه خروجی	نمونه ورودی
['U', 'D', 'R', 'U', 'U', 'U', 'L', 'U']	W W W W A W W W P W W . . W W S W W . W . P . W W . W . . A W W W W W W W W

حروف URDL به ترتیب حرکت رو به بالا، راست، پایین و چپ را نشان می‌دهند. بدیهی‌ست که جواب بهینه‌ی مسئله لزوماً یکتا نیست پس هر جواب بهینه‌ای درست است.

پیاده‌سازی مسئله

در این تمرین، شما باید مسئله را با سه روش جستجوی ناآگاهانه‌ی IDS، BFS، DFS و همچنین روش جستجوی آگاهانه‌ی A^* حل و پیاده‌سازی کنید. هنگام پیاده‌سازی الگوریتم‌ها به بخش سوالات نیز دقت کنید زیرا در آنها ایده‌ها و راهنمایی‌هایی برای پیاده‌سازی بهتر الگوریتم‌ها وجود دارند.

برای روش A^* ، حداقل سه heuristic مناسب معرفی و پیاده‌سازی کنید. دقت کنید که در نهایت الگوریتم A^* باید پاسخ بهینه را پیدا کند. بنابراین heuristic را از لحاظ admissible و consistent بودن بررسی کنید. در انتها، A^* weighted را با حداقل دو مقدار وزن و حالت بدون وزن اجرا کرده و آنان را با همدیگر مقایسه کنید.

الگوریتم A^* weighted

این الگوریتم نسخه‌ای از الگوریتم A^* است که در آن از یک وزن w برای تسریع فرایند استفاده می‌کنیم. تفاوت این الگوریتم با A^* در این است که خروجی تابع heuristic را در w ضرب می‌کنیم و بدین صورت وزن هزینه‌ی تخمین زده شده بیشتر از هزینه‌ی مسیر فعلی می‌شود. بنابراین فرایند جستجو حریصانه و سریع‌تر خواهد شد. در این الگوریتم تابع هزینه به شکل زیر تعریف می‌شود:

$$f(n) = w \times h(n) + g(n)$$

محدودیت زمانی اجرا

در جدول زیر زمان تقریبی برای اجرای هر الگوریتم (برحسب میلی‌ثانیه) روی هر نقشه‌ی نمونه که در اختیارتان قرار داده شده، آورده شده است. توجه کنید خانه‌هایی که با "-" مشخص شده بدین معنا می‌باشد که نیازی نیست آن الگوریتم روی آن نقشه در زمان معقولی به جواب برسد.

MAP10	MAP9	MAP8	MAP7	MAP6	MAP5	MAP4	MAP3	MAP2	MAP1	
2,000	20,000	1,000	100	100	5	50	10	30	1	BFS
30	50	50	5	1	5	1	1	1	1	DFS
-	-	-	10,000	-	100	40,000	2,000	1,000	10	IDS
10,000	50,000	5,000	200	400	5	50	20	10	1	A^*
8,000	40,000	4,000	150	300	3	30	10	5	1	$W A^*$

توجه داشته باشید که محدودیت‌های جدول بالا بر اساس نتایج دستیاران آموزشی تعیین شده‌اند و به عنوان یک راهنما برای ارزیابی عملکرد شما در نظر گرفته می‌شوند. با اینکه رعایت دقیق‌شان الزامی نیست، بهتر است آنها را به صورت تقریبی در نظر بگیرید و سعی کنید نتایج پیاده‌سازی خود را به آنان نزدیک کنید. در نهایت زمان و نتایج اجرای الگوریتم‌ها روی نقشه‌های مختلف را در گزارش ذکر کنید. همچنین هنگام تحویل پروژه نقشه‌های جدیدی تست خواهند شد که از نظر سختی مشابه نقشه‌های داده شده خواهند بود.

توضیحات کد

برای پیاده‌سازی این تمرین، دو نسخه در اختیار شما قرار داده شده است که هسته منطقی و الگوریتم‌های آنها یکسان است و می‌توانید هر کدام را که راحت‌تر هستید انتخاب کنید:

۱. نسخه‌ی Notebook (اولویت‌دار)

در این نوت‌بوک تمام بخش‌های مورد نیاز برای پیاده‌سازی و آزمایش الگوریتم‌های جستجو آماده شده‌اند و کافی است قسمت‌هایی که با #TODO مشخص شده‌اند را تکمیل کنید.

ابتدا باید ساختار state را تعریف کنید؛ هر state باید اطلاعات لازم برای نمایش یک گره در گراف جستجو را داشته باشد. سپس باید تابع next_state را پیاده‌سازی کنید که با دریافت state فعلی، board و یک action، state بعدی را محاسبه و برگرداند؛ این تابع باید رفتار مار در زمین بازی را مدیریت کند. در مرحله‌ی بعد الگوریتم‌های BFS، DFS، IDS، A* و Weighted A* را در توابع مربوطه پیاده‌سازی کنید. هر تابع باید مسیر پیدا شده و تعداد state های بازدید شده را برگرداند. در صورت عدم یافتن مسیر یا تجاوز از time_limit، لیست خالی برگردانید. توجه داشته باشید که حداقل سه تابع heuristic مناسب تعریف کنید و نام آن‌ها را به لیست heuristics اضافه کنید تا در رابط گرافیکی قابل انتخاب باشند. توابع render_notebook و show_path برای نمایش وضعیت بازی و اجرای مسیر گام‌به‌گام در نوت‌بوک آماده شده‌اند. همچنین تابع run_ui یک سرور WebSocket روی پورت ۸۷۶۵ راه‌اندازی کرده و فایل index.html را در مرورگر باز می‌کند تا بتوانید بازی را به‌صورت تعاملی مشاهده کنید. نوت‌بوک باید در محیط Jupyter اجرا شود و فایل‌های map.py، game.py و index.html در همان پوشه قرار داشته باشند.

لازم است برای تمام نقشه‌ها و الگوریتم‌ها از قابلیت “چاپ لاگ‌ها در کنسول سرور” استفاده کنید و خروجی‌های کنسول را در گزارش خود قرار دهید.

۲. نسخه‌ی Python Script (جایگزین)

در این نسخه، همان منطق بازی و الگوریتم‌های نوت‌بوک وجود دارند اما به‌صورت یک اسکریپت پایتون قابل اجرا در ترمینال یا رابط گرافیکی ارائه شده‌اند. کلاس‌های Map، Snake و Game در فایل game.py قرار دارند و منطق بازی را پیاده‌سازی می‌کنند. الگوریتم‌های جستجو و توابع heuristic در فایل اصلی تعریف شده‌اند و باید بخش‌های #TODO را تکمیل کنید. با اجرای فایل main.py، یک سرور WebSocket روی پورت ۸۷۶۵ راه‌اندازی شده و فایل index.html در مرورگر باز می‌شود. در رابط گرافیکی می‌توانید نقشه‌های مختلف را انتخاب کنید، مار را به‌صورت دستی با کلیدهای جهت‌دار کنترل کنید، الگوریتم‌های جستجو را اجرا کرده و مسیر تولید شده را به‌صورت بصری مشاهده کنید و همچنین برای A* و Weighted A*، heuristic و وزن موردنظر را انتخاب کنید. فایل main.py را اجرا کنید و فایل‌های map.py، game.py و index.html نیز باید در همان پوشه باشند.

توجه داشته باشید که برای تمام نقشه‌ها و الگوریتم‌ها، تصاویر (اسکرین‌شات) خروجی‌ها باید در گزارش قرار داده شوند.

سوالات

۱. در این مسئله تعریف state را چه می توان در نظر گرفت؟ بدیهی ترین تعریف برای آن نقشه ی بازی در هر لحظه است. ایراد این تعریف چیست؟ چگونه میتوان آن را رفع کرد؟
 ۲. نحوه ی مدل سازی مسئله با یک گراف را توضیح دهید و گره های مجاور در آن را توصیف کنید.
 ۳. آیا در الگوریتم های جستجو بایستی تمام حالات ممکن که میتوانیم در قدم بعدی حرکت کنیم را گسترش (expand) دهیم؟ این کار چه نتایجی دارد؟ چگونه می توان این مسئله را بهبود داد و فضای سرچ را هرس کرد؟
 ۴. هر یک از الگوریتم های پیاده سازی شده را توضیح دهید. سپس تفاوت و مزیت های هر یک نسبت به دیگری (مثلا مدت زمان جستجو، میزان حافظه ی مصرفی و تعداد state های دیده شده) را توضیح دهید و عنوان کنید که کدام الگوریتم ها جواب بهینه می دهند و چرا. در انتها نتایج اجرای خود را با توضیحات داده شده تطبیق دهید.
 ۵. با توجه به پاسخ سوال قبل و مزیت های هر کدام از DFS و BFS نسبت به همدیگر، چگونه IDS این دو را ترکیب می کند و چه نقاط ضعفی از آنان را می پوشاند.
 ۶. یک ایده ساده برای heuristic، استفاده از مجموع فاصله منتهن مار از تمام سیب هاست. مشکل این heuristic چیست؟ چگونه میتوان آن را اصلاح کرد؟
 ۷. در مسئله جستجوی ما، برای یک heuristic آیا admissible بودن کافی ست یا حتما باید consistent هم باشد؟
 ۸. توابع heuristic خود را توضیح دهید.
- پاسخ این سوالات و مواردی که پیش تر عنوان شد را در گزارش خود بیاورید.

نکات پایانی

- در شرایط فعلی که دسترسی به اینترنت بین الملل نداریم، می توانید پکیج های پایتون مورد نیاز خود را از این **mirror** دریافت کنید. برای مثال کافی ست دستور زیر را اجرا کنید:

```
pip install -i https://mirror-pypi.runflare.com/simple websockets
```


در صورت ابهام این **لینک** راهنما را مطالعه کنید.
- کد شما باید به نحوی زده شود که نتایج با پارامترهای مختلف قابلیت باز تولید داشته باشند.
- در این تمرین اولویت با نسخه ی Notebook است. اگر مشکلی در نصب داشتید یا به هر دلیلی اجرای آن ممکن نبود، می توانید از اسکریپت پایتون استفاده کنید.

- توضیحات مربوط به هر بخش از تمرین را خلاصه و مفید در گزارش ذکر کنید. حجم توضیحات هیچ تاثیری در نمره نخواهد داشت و تحلیل شماست که ارزشمند است.
- فایل‌های خود که شامل گزارش و کد می‌باشند را در قالب یک فایل فشرده با فرمت `AI_CA1_[stdNum].zip` در سامانه‌ی `elearn` بارگذاری کنید. به طور مثال `AI_CA1_810103127.zip`.
- توجه کنید این تمرین باید به صورت تک‌نفره انجام شود و پاسخهای ارائه شده باید نتیجه‌ی فعالیت فرد نویسنده باشد. در صورت مشاهده تقلب به همه‌ی افراد مشارکت‌کننده، نمره‌ی ۱۰۰- داده شده و به استاد نیز گزارش می‌گردد. همچنین نوشته شدن کدها توسط هوش مصنوعی نیز بررسی میشود!
- سوالات خود را از طراحان تمرین از طریق ایمیل یا گروه درس بپرسید.

موفق باشید.